Assignment 6: Enterprise Data Architecture and Modelling

Scenario: a marketplace and an online retailer that need one analytics platform for sales, delivery and customer reporting. Datasets: Olist Brazilian e-commerce (Kaggle) and UCI Online Retail.

1. Characteristics of the modern architecture applied here

• **Separate storage and compute.** Raw and curated data sit in object storage (S3); a warehouse serves queries. Each can scale on its own.

• **Layered (medallion) processing.** Bronze keeps the data as received, silver holds clean and typed data, gold holds the analysis model. Any layer can be rebuilt from the one before it.

• **Mixed sources in one model.** The two datasets have different keys and coverage, so they share date and location but keep their own customer and product dimensions.

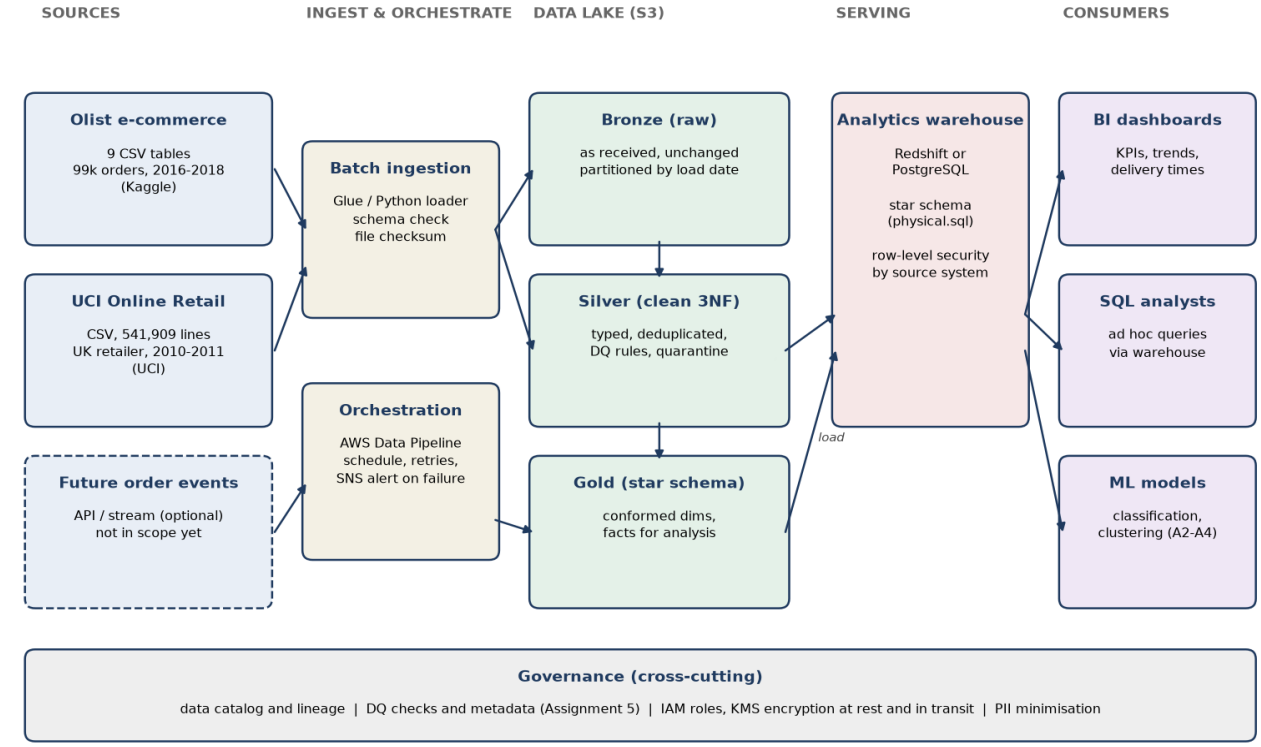
• **Governance built in.** Catalog, lineage, data-quality checks and access control (Assignment 5) are part of the design, not added later.

• **Open for new sources.** A streaming or API source can be added at ingestion without changing gold tables.

2. Enterprise data architecture

Figure 1 shows the flow. Batch loaders copy the CSV files into bronze, with a schema check and file checksum. AWS Data Pipeline schedules the steps and sends an alert on failure. Silver applies the data-quality rules from Assignment 5 and quarantines bad rows. Gold builds the star schema, which is loaded into the warehouse for BI, SQL and model training. A streaming source is shown dashed because it is not in scope.

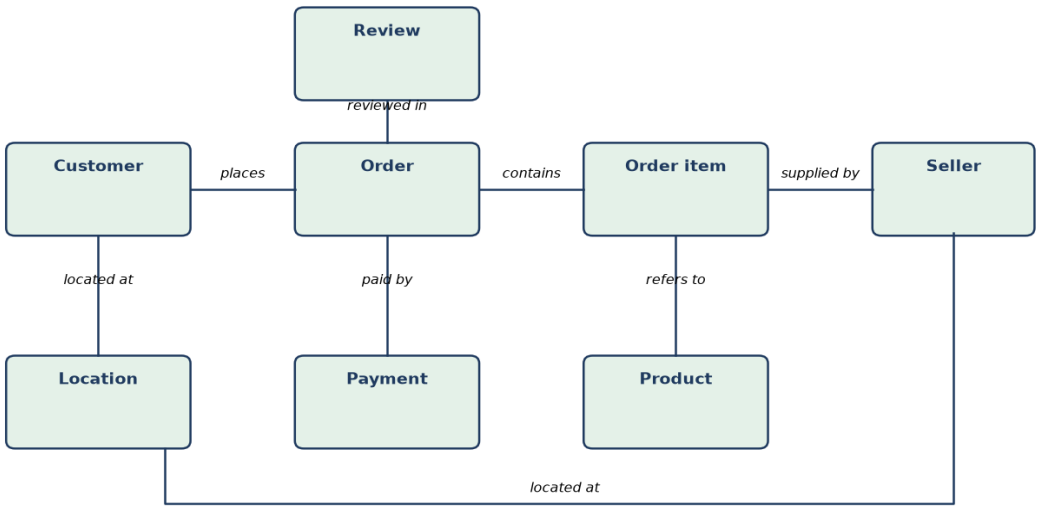
Figure 1. Enterprise data architecture (batch, medallion layers)



3. Data models

Conceptual. Eight business concepts: Customer, Order, Order item, Product, Seller, Payment, Review and Location (Figure 2). The retail data maps Invoice to Order and StockCode to Product. It has no Seller, Payment or Review, so those are empty for that source. Full relationships and cardinalities are in *models/conceptual.md*.

Figure 2. Conceptual model



Retail mapping: Invoice = Order, Invoice line = Order item, StockCode = Product.
Retail has no Seller, Payment or Review, so those entities are empty for that source.

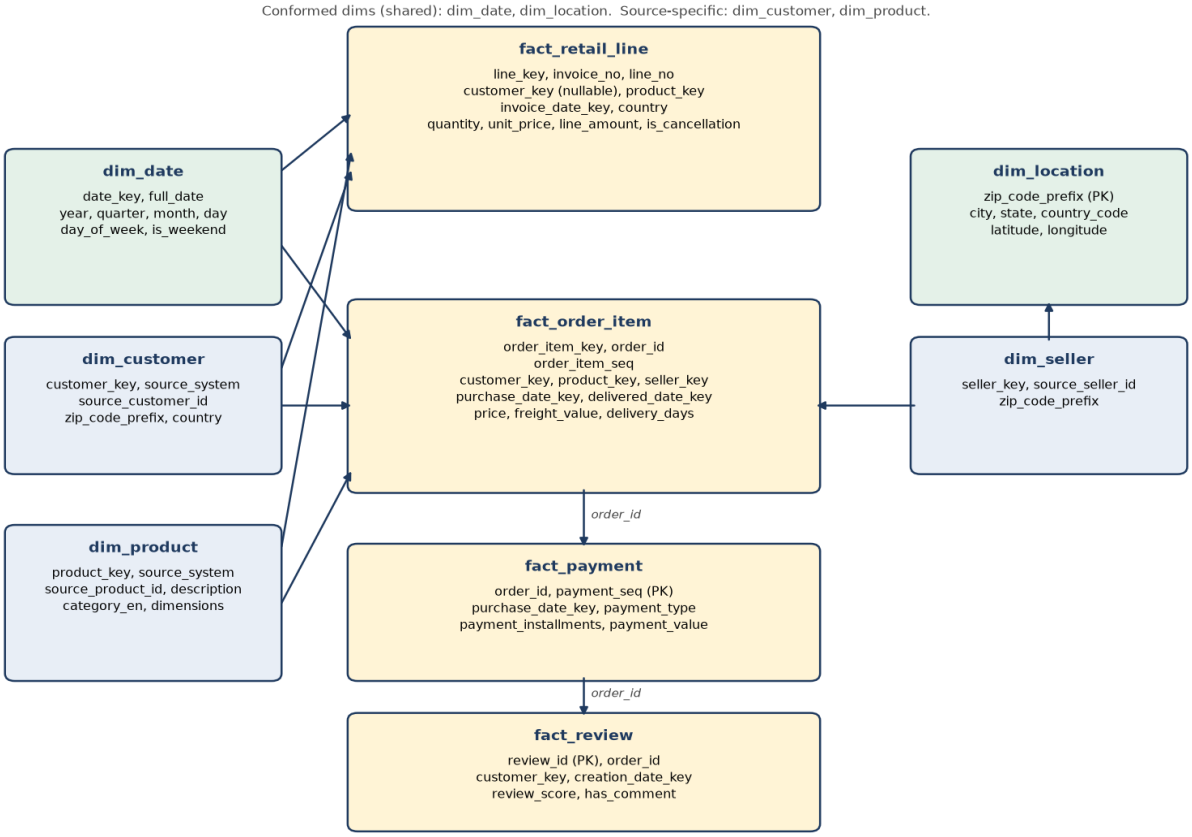
Logical. Normalised to third normal form in silver, with attributes, keys, nullability and descriptions in *models/logical_model.csv*. Summary:

Logical entity (silver, 3NF)	Source	Primary key	Attributes
customer	olist	customer_id	3

Logical entity (silver, 3NF)	Source	Primary key	Attributes
orders	olist	order_id	7
product	olist	product_id	6
product_category	olist	category_name_pt	2
seller	olist	seller_id	2
order_item	olist	order_id, order_item_id	7
order_payment	olist	order_id, payment_sequential	5
order_review	olist	review_id	5
location	olist	zip_code_prefix	5
retail_invoice	uci_retail	invoice_no	4
retail_line	uci_retail	invoice_no, line_no	5
retail_product	uci_retail	stock_code	2

Physical. Gold is a star schema on Redshift, written as portable SQL in *models/physical.sql* and tested in SQLite (Figure 3). Redshift distribution and sort keys are noted in comments.

Figure 3. Physical model: gold star schema (see models/physical.sql)



Physical table	Columns	Role
dim_customer	5	dimension
dim_date	8	dimension
dim_location	6	dimension
dim_product	9	dimension
dim_seller	3	dimension
fact_order_item	12	fact
fact_payment	6	fact
fact_retail_line	11	fact
fact_review	6	fact

4. Key design decisions and rationale

Decision	Rationale
Bronze, silver and gold layers	Raw data is kept, so any later fix can be replayed. Each layer has one job, which makes lineage easy to follow.
Silver in 3NF, gold as a star	3NF keeps each fact stored once and makes integrity checks simple. A star lets BI tools write short, fast queries.
Conformed date and location; source-specific customer and product	Dates and places mean the same thing in both datasets. Olist and UCI product codes do not match, so forcing one dimension would give false matches.
Olist customer key = customer_unique_id	The order-level customer_id changes for each order. The unique id identifies the person across orders.
Fact grains: order item, payment, review	Revenue is measured per item. Payments are many per order and reviews are zero or one per order, so each gets its own fact. This avoids double-counting order totals.
Nullable customer_key in fact_retail_line	A large share of UCI rows have no CustomerID. Dropping them would understate revenue, so the rows are kept and reported as unknown customer.
Cancellations and returns kept, flagged	Negative quantities and cancelled invoices (C prefix) are flagged, not deleted, so net sales can be reconciled.
Review text excluded from gold	Free text can hold personal data. Minimisation keeps it in silver, under access control, and out of the analytics layer.
Surrogate integer keys, source ids kept	Stable keys across source changes, while the original ids stay for tracing back to bronze.
Distribution keys on order_id; small dims replicated	Order items, payments and reviews join locally on Redshift. Small date and location tables are copied to every node.
Batch ingestion now; streaming left as an extension	The two datasets are historical files loaded in batches. A stream can be added at ingestion without changing gold.

5. Limitations

• The design was built from each dataset's documented schema. The CSV files were not loaded into this environment, so row counts and null rates are taken from the dataset descriptions, not measured.

• The DDL was run in SQLite, not on Redshift. Platform-specific types and distribution keys are untested.

• No AWS resources were created, so there are no console screenshots of the security settings.

6. References

• Olist, Brazilian E-Commerce Public Dataset. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

• Chen, D. Online Retail. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/352/online+retail

• Kimball, R. and Ross, M. The Data Warehouse Toolkit, 3rd ed. Wiley, 2013 (star schema and grain).

• AWS. Well-Architected Framework, Data Analytics Lens. https://docs.aws.amazon.com/wellarchitected/latest/analytics-lens/

• Amazon Redshift Database Developer Guide: distribution styles and sort keys.